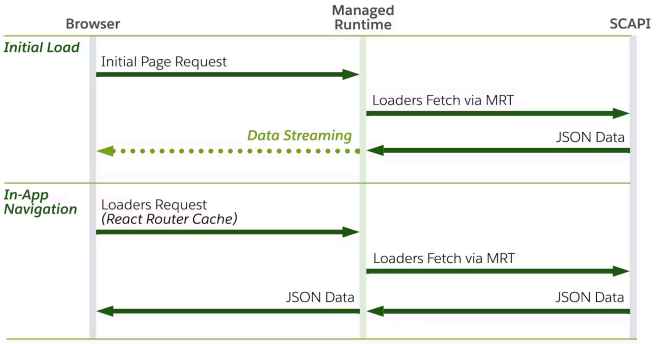
Data Fetching

Storefront Next is built on React Router, leveraging its Framework Mode to provide a structured foundation for routing and data handling. As such, Storefront Next represents a **server-rendered single-page application** (SPA). This application model means that only the initial navigation request to a route is processed and responded to by the server. All subsequent client-side navigation requests are routed on the client and only trigger requests for data or additionally required assets, or both. Server-side rendering (SSR) ensures fast initial load times while client-side navigation and rendering (CSR) eliminates full page reloads after hydration, combining the strengths of SSR and CSR without their respective tradeoffs.



Storefront Next uses these data loading components:

- **Loaders:** Server-side functions that fetch data before component rendering.
- **Actions:** Server-side functions that handle data mutations triggered by form submissions or programmatic calls.
- **Fetchers:** Client-side functions that enable loading data from or submitting data to routes without causing navigation. Fetchers can call loaders (for reads) or actions (for writes) on any route, making them ideal for in-page interactions.
- **Middlewares:** Functions that run in a pipeline before loaders and actions and allow intercepting navigation requests before a route renders.
- **Cookies and Sessions:** Server-side mechanisms for persisting state across requests, such as user preferences, authentication tokens, and flash messages.

Dependencies

Contents

`react`	19.2.0	React 19 with `use()` hook

Paradigms

Server-Load Everything

One of the many strengths of React Router lies in its highly flexible mechanisms for controlling and calibrating data retrieval and data flows within an app. While it's technically possible to implement complex, server/client-segregated data flows, we made the deliberate architectural decision for Storefront Next to promote a **server-load everything** paradigm.

> **Important**
>
> In our proposed architecture, Managed Runtime (MRT) isn't only used as a simple data proxy, but acts as ***the* data orchestration layer**. Using React Router's server data loading functionality, we're able to aggregate parallel and sequential SCAPI requests into a single request to MRT and progressively stream the response to the client. This ultimately means that all API requests are executed on the server (that is, MRT).

A solid understanding of this paradigm is suggested, as it directly impacts the structure and bundling of the app code, as well as overarching aspects such as performance, authentication, security, and SEO.

Route-Level Data Fetching

In Storefront Next, we promote route-level data fetching via loaders, as they are the only mechanism that guarantees data is fetched before component rendering on the server.

> **Note**
>
> **Component-level data fetching** (for example, `useEffect` or direct `fetch` calls within components) isn't recommended for initial page loads, as data is absent during SSR, resulting in degraded SEO and slower perceived performance. All of these client-side data fetching strategies should only be considered for interaction-driven data scenarios.

Data Classification

Another pillar of an effective data fetching strategy is classifying data by its route-level relevance. This strategy requires answering three questions per route.

content, but also negotiable for above-the-fold content in case of visually stable content/layouts.)

3. What data is interaction-driven? (Outside the initial data fetching lifecycle; fetched on user action.)

Critical Data

Critical data comprises all information required to produce a complete, semantically correct, and layout-stable initial HTML response. This data includes above-the-fold content that determines Largest Contentful Paint (LCP), layout-defining attributes such as image dimensions that affect Cumulative Layout Shift (CLS), and all SEO-relevant artifacts such as accurate title, meta description, canonical links, structured data, and correct HTTP status codes (for example, 200, 301, 404), and so on. It also includes any data necessary to render the correct document state for crawlers and social previews. Data is considered critical if its absence delays meaningful paint, changes initial layout, misrepresents document semantics, or produces an incorrect status code.

Non-Critical Data

Non-critical data comprises information that doesn't affect initial render completeness, layout stability, document semantics, or HTTP correctness. It doesn't negatively impact loading metrics like LCP, or the visual stability of the above-the-fold structure, and isn't required for accurate indexing or link previews. This category includes below-the-fold content, related items, recommendations, progressive personalization, analytics payloads, and enrichment UI elements. Such data can be deferred, streamed, or lazy-loaded without compromising performance metrics, crawlability, or document validity. Learn more about visual stability here.

Interaction-Driven Data

Interaction-driven data is fetched in response to user actions, such as clicking a button, submitting a form, or triggering other UI interactions. Unlike route-level data fetching that occurs during navigation, interaction-driven data is requested after the initial page render.

> **Note**
>
> The React Router framework provides loaders for data fetching and actions for interaction-driven data mutations, with middlewares handling cross-cutting concerns (for example, authentication, logging, caching), and fetchers enabling interaction-driven data updates without navigation.

Loaders

Loaders are exported functions in route modules that fetch data before (or during) the component tree renders.

The individual values within the returned record can themselves be either concrete values or Promises. In React Router, unresolved Promises inside the returned object are handled natively and can be consumed in combination with React `<Suspense/>`.

This enables fine-grained control over rendering behavior by separating:

- **Critical Data:** values awaited within the loader before returning the object.
- **Non-Critical Data:** Promises returned as values, resolved later and rendered within `<Suspense/>` boundaries.

Loader Arguments

Argument	Type	Description
`request`	Request	Standard Fetch API Request object.
`params`	Object	Route parameters from URL.
`context`	RouterContextProvider	Shared context from middleware.

Server Loaders

While React Router also provides the concept of client loaders, we recommend the consistent use of server loaders in accordance with our server-load everything paradigm. Server loaders get invoked during server-side rendering (SSR), but also to fetch data during subsequent client-side navigation requests.

Server loaders offer several advantages and enhancements:

- **Security** can be enhanced by keeping API credentials and sensitive business logic server-side, never exposing them to the client bundle. Additionally, server loaders enable direct access to backend services, databases, and internal APIs without CORS concerns or public endpoint exposure.
- Perceived **performance** can benefit from server-side rendering as Core Web Vitals metrics like LCP (Largest Contentful Paint) and TTI (Time to Interactive) can improve significantly by delivering pre-rendered HTML. CLS (Cumulative Layout Shift) improvements require explicit layout stability measures like reserved space for dynamic content (for example, via skeletons).
- **Client bundle size** can remain minimal since data fetching code and dependencies don't have to get shipped to the browser if kept out of the client module graph.
- **SEO** can benefit from fully-rendered HTML with data already present in the initial response, enabling bots to crawl complete content without having to execute JavaScript. Dynamic meta tags (title, description, Open Graph) can be populated with actual data, improving social sharing.

function returns an object containing the two resolved data portions.

```
// src/routes/product.$productId.tsx
import type { LoaderFunctionArgs } from "react-router";

export async function loader({ request, params, context }
  const { productId } = params;

  // Fetch data from two external APIs
  const productPromise = fetch(`https://api.example.com/p
    r.json(),
  );
  const recommendationsPromise = fetch(
    `https://api.example.com/recommendations/${productId}
  ).then((r) => r.json());

  // Parallelize data fetching
  const [product, recommendations] = await Promise.all([p

  return {
    product,
    recommendations,
  };
}
```

Example: Critical Data - Handling 404 Responses for SEO

When a requested resource doesn't exist, returning a proper 404 HTTP status code is critical for SEO. Search engines distinguish between valid pages and missing content to maintain accurate indexes and avoid crawl budget waste.

This code example shows how to throw a 404 response when a product isn't found. The thrown `Response` is caught by the framework and returned with the correct HTTP status code, ensuring search engines and social media crawlers receive proper signals.

```
// src/routes/product.$productId.tsx
import type { LoaderFunctionArgs } from "react-router";

export async function loader({ request, params, context }
  try {
    const { productId } = params;
    const product = await fetch(`https://api.example.com/
      r.json(),
    );
    return {
      product,
    };
  } catch {
    // Throw 404 Response if product doesn't exist
    throw new Response("Product not found", { status: 404
  }
}
```

Tip

handled by the closest `ErrorBoundary` in your route hierarchy catches and handles the thrown `Response`, enabling you to render custom 404 pages.

Example: Non-Critical Data - Streaming and Progressive Loading

React Router awaits route loaders before rendering route components. To unblock the loader for non-critical data, simply return a Promise instead of awaiting it in the loader.

Non-critical data enables **progressive rendering** by prioritizing above-the-fold content while deferring below-the-fold elements. This improves **perceived performance** by showing users meaningful content faster, reducing time-to-interactive. For **SEO**, critical data like product titles and descriptions are immediately available in the initial HTML, while supplementary content like reviews or recommendations can stream in afterward. This pattern optimizes the balance between fast initial page loads and comprehensive content delivery.

This code example defines a loader that demonstrates both data deferral and Promise chaining. The goal is to maximize performance by initiating fast and independent fetches immediately, while organizing dependent fetches to run as soon as their prerequisite data is available.

```
// src/routes/product.$productId.tsx
import type { LoaderFunctionArgs } from "react-router";

export async function loader({ request, params, context }
  const { productId } = params;

  // Needs to be fast as it will both block the initial r
  // is required for chained data fetching
  const product = fetch(`https://api.example.com/products

  // Dependent data requires chaining
  const category = product
    .then((product) => fetch(`https://api.example.com/cat
    .then((r) => r.json());

  // Independently resolvable data
  const recommendations = fetch(`https://api.example.com/
    r.json(),
  );

  return {
    product: await product, // <-- Critical: Await the Pr
    category, // <-- Non-Critical: Return the Promise
    recommendations, // <-- Non-Critical: Return the Prom
  };
}
```

Example: Consuming Loader Data

Components receive loader data and can either use React 19's `use()` hook or React Router's `<Await/>` component to unwrap promises. This code example shows how to consume critical

that suspend within it.

```tsx
// src/routes/product.$productId.tsx
import { Suspense } from "react";
import Category, { CategorySkeleton } from "@/components/
import Recommendations, { RecommendationsSkeleton } from

type ProductPageData = {
  product: Product,
  category: Promise<Category>,
  recommendations: Promise<Recommendation[]>,
};

export default function ProductPage({
  loaderData: { product, category, recommendations },
}: {
  loaderData: ProductPageData,
}) {
  return (
    <>
      <h1>{product.name}</h1>
      <p>{product.description}</p>

      <Suspense fallback={<CategorySkeleton />}>
        <Category promise={category} />
      </Suspense>

      <Suspense fallback={<RecommendationsSkeleton />}>
        <Recommendations promise={recommendations} />
      </Suspense>
    </>
  );
}
```

Tip

You can also use the `useLoaderData` hook to retrieve the data.

```tsx
import { useLoaderData } from "react-router";

export default function ProductPage() {
  const loaderData = useLoaderData<ProductPageData>();
  // ...
}
```

For details on how loader data integrates with the broader state model, including `useLoaderData`, `useRouteLoaderData`, and middleware context as state, see State Management. For visual feedback during data loading, see Loading States.

SCAPI Request Shape

SCAPI endpoints expose several knobs that control response size and cacheability, such as `expand`, `select`, `limit` / `offset`, and similar

in the SSR response. Over-fetching adds bytes to TTFB, inflates the streamed HTML, and increases hydration cost. Audit each loader and ask which fields the route actually renders. Drop the rest.

- Cache TTL. SCAPI caches responses per unique request URL, including the full query string. Adding or removing a parameter creates a separate cache entry. More importantly, fields with shorter TTLs (for example, real-time inventory or pricing) pull the entire cached response onto their shorter schedule when included inline. Prefer fetching short-TTL data separately via a non-critical deferred loader rather than mixing it into critical, long-cacheable payloads. See Expand Parameter Impact on Cache Hit Rates for the underlying caching behavior.

Actions

Action functions handle data mutations, such as form submissions, updates, or deletions, the counterpart to loaders. While loaders handle read operations, actions handle writes. A GET/POST/PUT/DELETE distinction is a useful mental model, though React Router routes requests by navigation intent rather than HTTP method alone.

Action Arguments

Argument	Type	Description
`request`	Request	Standard Fetch API Request object.
`params`	Object	Route parameters from URL.
`context`	RouterContextProvider	Shared context from middleware.

Server Actions

Comparable to server loaders, React Router also provides the concept of client actions. In line with our server-load everything paradigm, we recommend server actions exclusively. Server actions are functions that execute solely on the server, ensuring sensitive mutation logic, such as database writes or authentication checks, never reaches the client.

Action Return Pattern

Always return `data(payload, init?)` from `react-router` – never `Response.json(...)`. This:

- Preserves the HTTP status code so CDNs, server logs, and client-side `fetcher.formMethod` checks behave correctly.
- Keeps the payload type inferable, so consumers can use `useFetcher<typeof action>()` and get full type-safe access to `fetcher.data`.

```
import { data } from 'react-router';
import type { Route } from './+types/action.example';

/** Response shape returned by the example action. Export
export type ExampleResponse = {
  success: boolean;
  error?: ActionError;
};

export async function action({
  request,
  context,
}: Route.ActionArgs): Promise<ReturnType<typeof data<Exam
  // ...
  return data({ success: true });
}
```

Consumers then bind the fetcher to the action:

```
import type { action as exampleAction } from "@/routes/ac

const fetcher = useFetcher<typeof exampleAction>();
// fetcher.data is typed as ExampleResponse
```

For an action that may dispatch to one of several routes (for example, a single fetcher submitting to add/remove/update endpoints), use a union: `useFetcher<typeof addAction | typeof removeAction>()`.

Example: Interaction-Driven Data - Newsletter Signup Action

This example demonstrates interaction-driven form submission with validation and error handling via an action function.

```
// src/routes/newsletter.tsx
import { type ActionFunctionArgs, data, Form, useActionDa

export async function action({ request }: ActionFunctionA
  const formData = await request.formData();
  const email = formData.get('email') as string;

  // Validate email
  if (!email || !email.includes('@')) {
    return data(
      { error: 'Please enter a valid email address' },
      { status: 400 },
    );
  }

  // Call API/service
  const response = await fetch('https://api.example.com/r
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email }),
  });

  if (!response.ok) {
    return data(
```

```
}

export default function NewsletterPage() {
  const actionData = useActionData<typeof action>();

  return (
    <Form method="post">
      <input type="email" name="email" required />
      <button type="submit">Subscribe</button>
      {actionData?.error && <p className="error">{actionD
      {actionData?.success && <p className="success">Subs
    </Form>
  );
}
```

For details on how action return values integrate with the state model (optimistic UI, `fetcher.data`, `useActionState`), see State Management.

Fetchers

Fetchers are React Router's mechanism for triggering loaders or actions outside of navigation, enabling data fetches and mutations without changing the current route or URL. Unlike standard navigation, multiple fetchers can run concurrently and independently, each tracking their own submission state. This makes them well-suited for use cases such as inline form submissions, optimistic UI updates, or background data refreshes.

Example: Interaction-Driven Data - Newsletter Signup Fetcher

This example demonstrates the same interaction-driven form processing as the previous example, but this time using a fetcher (via the `useFetcher` hook).

```
// src/routes/newsletter.tsx
import { data, type ActionFunctionArgs, useFetcher } from

export async function action({ request }: ActionFunctionA
  const formData = await request.formData();
  const email = formData.get("email") as string;

  // Validate email
  if (!email || !email.includes("@")) {
    return data({ error: "Please enter a valid email addr
  }

  // Call API/service
  const response = await fetch("https://api.example.com/r
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email }),
  });

  if (!response.ok) {
    return data({ error: "Subscription failed" }, { statu
  }
  return data({ success: true });
```

```
29   return (
30     <fetcher.Form method="post">
31       <input type="email" name="email" required />
32       <button type="submit" disabled={fetcher.state === '
33         {fetcher.state === "submitting" ? "Subscribing...
34       </button>
35       {fetcher.data?.error && <p className="error">{fetch
36       {fetcher.data?.success && <p className="success">Su
37     </fetcher.Form>
38   );
39 }
```

For details on how `fetcher.state` and `fetcher.data` integrate with the state model, see State Management. For visual feedback patterns based on `fetcher.state`, see Loading States.

Resource Routes

Resource routes are specialized routes that don't render UI components but instead function as API endpoints within your app. Unlike traditional UI routes that export both a loader/action and a component, resource routes typically export only loaders, actions, or both. This makes them ideal for encapsulating server-side business logic that can be called from anywhere in your app.

Resource routes bridge the gap between loaders and actions by providing a dedicated location for reusable data operations. They leverage the same loader and action patterns you've learned, but organize them as callable endpoints rather than navigation destinations.

When to use Resource Routes

Resource routes are particularly useful for:

- **Encapsulated business logic:** Complex operations that benefit from being isolated in dedicated route modules, such as basket management, wishlist operations, or multi-step workflows
- **Reusable endpoints:** Data operations that multiple components across different routes need to access without duplicating code
- **Non-navigational mutations:** Actions that modify data but shouldn't trigger navigation, such as adding items to basket, toggling favorites, or updating preferences
- **Direct SCAPI calls:** Generic API proxy routes (for example, `resource/api/client/*`) that forward requests to SCAPI without requiring SCAPI client libraries on the client

Defining Resource Routes

Resource routes follow the same patterns as regular routes but typically don't export a default component. They can be prefixed with `resource/` to distinguish them from UI routes.

Example: Interaction-Driven Data - Newsletter Signup Action and Fetcher

```
// src/routes/resource.newsletter-signup.ts
import { data, type ActionFunctionArgs } from "react-rout

// Resource route action - no component exported
export async function action({ request }: ActionFunctionA
  const formData = await request.formData();
  const email = formData.get("email") as string;

  // Validate email
  if (!email || !email.includes("@")) {
    return data({ error: "Please enter a valid email addr
  }

  // Call API/service
  const response = await fetch("https://api.example.com/r
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email }),
  });

  if (!response.ok) {
    return data({ error: "Subscription failed" }, { statu
  }
  return data({ success: true });
}
```

```
// src/components/newsletter-form.tsx
import { useFetcher } from "react-router";

export default function NewsletterForm() {
  const fetcher = useFetcher();

  return (
    <fetcher.Form method="post" action="/resource/newslet
      <input type="email" name="email" required />
      <button type="submit" disabled={fetcher.state === '
        {fetcher.state === "submitting" ? "Subscribing...
      </button>
      {fetcher.data?.error && <p className="error">{fetch
      {fetcher.data?.success && <p className="success">Su
    </fetcher.Form>
  );
}
```

Example: Interaction-Driven Data - Basket Mutation Action

This example shows a resource route that encapsulates complex business logic for adding items to a shopping basket.

```
// src/routes/resource.basket.add.ts
import { data, type ActionFunctionArgs } from "react-rout
import { addToBasket } from "@/lib/basket";
import { validateInventory } from "@/lib/inventory";

export async function action({ request, context }: Actior
  const formData = await request.formData();
  const productId = formData.get("productId") as string;
```

```
    return data({ error: "Insufficient inventory" }, { st
  }

  // Add to basket via API
  const basket = await addToBasket(context, productId, qu

  return data({ basket });
}
```

```tsx
// src/components/add-to-basket-button.tsx
import { useFetcher } from "react-router";

export default function AddToBasketButton({
  productId,
  quantity,
}: {
  productId: string,
  quantity: number,
}) {
  const fetcher = useFetcher();

  const handleAddToBasket = () => {
    fetcher.submit({ productId, quantity }, { method: "po
  };

  return (
    <button onClick={handleAddToBasket} disabled={fetcher
      {fetcher.state === "submitting" ? "Adding..." : "Ad
    </button>
  );
}
```

Example: Generic API Proxy Route

For direct API calls without client-side API libraries, you can use a generic proxy route pattern.

```tsx
// src/routes/resource.api.$.ts
import type { LoaderFunctionArgs } from "react-router";

// Generic proxy for GET requests to your API
export async function loader({ request, params }: LoaderF
  const apiPath = params["*"]; // Captures the wildcard p
  const url = new URL(request.url);

  // Forward to API with credentials
  const response = await fetch(`https://api.example.com/$
    headers: {
      Authorization: `Bearer ${process.env.API_TOKEN}`,
      "Content-Type": "application/json",
    },
  });

  return response;
}

// Generic proxy for POST/PUT/DELETE requests to your API
export async function action({ request, params }: LoaderF
  const apiPath = params["*"];
```

```
      Authorization: `Bearer ${process.env.API_TOKEN}`,
      "Content-Type": "application/json",
    },
    body,
  });

  return response;
}
```

```tsx
// src/components/product-reviews.tsx
import { useEffect } from "react";
import { useFetcher } from "react-router";

export default function ProductReviews({ productId }: { p
  const fetcher = useFetcher();

  useEffect(() => {
    // Load reviews on mount via generic API proxy
    fetcher.load(`/resource/api/products/${productId}/rev
  }, [productId]);

  if (fetcher.state === "loading") {
    return <div>Loading reviews...</div>;
  }
  if (!fetcher.data) {
    return null;
  }
  return (
    <div>
      {fetcher.data.reviews.map((review) => (
        <div key={review.id}>{review.text}</div>
      ))}
    </div>
  );
}
```

Middlewares

React Router middleware consists of functions that intercept navigation requests before a route renders, enabling cross-cutting concerns, such as authentication, redirects, or logging, to be applied declaratively at the routing layer, upstream of loaders and actions.

Defining Middleware

```tsx
import type { MiddlewareFunction } from "react-router";

export const middleware: MiddlewareFunction<Response>[] =
  loggingMiddleware,
  appConfigMiddleware,
  authMiddleware,
];

export const clientMiddleware: MiddlewareFunction<Record<
  appConfigMiddlewareClient,
  authMiddlewareClient,
```

This code example defines a logging middleware function named `loggingMiddleware`. It's designed to run before any route loaders or other middleware in the chain, specifically to log request information and measure response times.

```
// src/middlewares/logging.server.ts
import type { MiddlewareFunction } from "react-router";

export const loggingMiddleware: MiddlewareFunction<Respor
  // Before: Log incoming request
  const startTime = Date.now();
  const url = new URL(request.url);
  console.log(`[${request.method}] ${url.pathname}${url.s

  // Execute next middleware / loader
  const response = await next();

  // After: Log response time and status
  const duration = Date.now() - startTime;
  console.log(`[${request.method}] ${url.pathname} - ${re

  return response;
};
```

Context System

Middlewares can store data in a shared context, which loaders and actions can access. The following code example shows this fundamental pattern. It uses the `createContext` utility provided by React Router to establish a communication channel that bypasses global state or complex dependency injection.

```
// Creating a context key
import { createContext, type LoaderFunctionArgs } from "r
export const requestMetricsContext = createContext<{ star

// In middleware: set context
export const loggingMiddleware: MiddlewareFunction<Respor
  context.set(requestMetricsContext, { startTime: Date.no
  // ...
};

// In loader: read context
export function loader({ context }: LoaderFunctionArgs) {
  const metrics = context.get(requestMetricsContext);
  // Use metrics data for performance tracking
}
```

For details on how middleware context functions as a state concept (request-scoped dependency injection vs. React Context API), see State Management.

Cookies and Sessions

`Cookie` header in loaders and actions, and written via `Set-Cookie` response headers. Because cookies travel with every HTTP request, they're available on the server during SSR without client-side synchronization, `localStorage` workarounds, or hydration mismatches.

Cookies

`createCookie` defines a reusable, typed cookie object with sensible defaults for attributes like `httpOnly`, `sameSite`, and `maxAge`. The cookie is read in a `loader` and written in an `action`.

```ts
// src/cookies.server.ts
import { createCookie } from "react-router";

export const userPrefs = createCookie("user-prefs", {
  path: "/",
  sameSite: "lax",
  httpOnly: true,
  secure: true,
  maxAge: 604_800, // one week
});
```

```tsx
// src/routes/home.tsx
import { type ActionFunctionArgs, data, Form, type Loader
import { userPrefs } from "@/cookies.server";

export async function loader({ request }: LoaderFunctionA
  const cookieHeader = request.headers.get("Cookie");
  const cookie = (await userPrefs.parse(cookieHeader)) ||
  return { showBanner: cookie.showBanner ?? true };
}

export async function action({ request }: ActionFunctionA
  const cookieHeader = request.headers.get("Cookie");
  const cookie = (await userPrefs.parse(cookieHeader)) ||
  const formData = await request.formData();

  if (formData.get("bannerVisibility") === "hidden") {
    cookie.showBanner = false;
  }

  return data({ ok: true }, { headers: { "Set-Cookie": av
}

export default function Home({ loaderData }: { loaderData
  return (
    <div>
      {loaderData.showBanner && (
        <div>
          <p>Don't miss our sale!</p>
          <Form method="post">
            <input type="hidden" name="bannerVisibility"
            <button type="submit">Dismiss</button>
          </Form>
        </div>
      )}
      <h1>Welcome!</h1>
```

`useEffect`, no SSR mismatch), writes it via the `action` with a `Set-Cookie` header, and triggers automatic revalidation so the UI reflects the new state immediately.

Sessions

For structured, server-managed state, such as authentication tokens or multi-field user profiles, React Router provides `createCookieSessionStorage`. A session storage object wraps cookie handling with `getSession`, `commitSession`, and `destroySession` helpers.

```ts
// src/sessions.server.ts
import { createCookieSessionStorage } from "react-router"

type SessionData = {
  userId: string;
};

type SessionFlashData = {
  error: string;
};

export const { getSession, commitSession, destroySession
  SessionData,
  SessionFlashData
>({
  cookie: {
    name: "__session",
    httpOnly: true,
    maxAge: 60 * 60 * 24, // 1 day
    path: "/",
    sameSite: "lax",
    secrets: ["s3cret1"],
    secure: true,
  },
});
```

```ts
// In a loader or action
import { getSession, commitSession } from "@/sessions.ser

export async function loader({ request }: Route.LoaderArg
  const session = await getSession(request.headers.get("C
  const userId = session.get("userId");
  // ...
}

export async function action({ request }: Route.ActionArg
  const session = await getSession(request.headers.get("C
  session.set("userId", "abc123");

  return data({ ok: true }, { headers: { "Set-Cookie": av
}
```

Sessions support flash data, values that exist only until the next read. That's useful for one-time messages like "Login successful" or validation errors across redirects.

All downstream loaders and actions then access the pre-parsed session without repeating the boilerplate.

```ts
// src/contexts.ts
import { createContext, type Session } from 'react-router

type SessionData = { userId: string };
type SessionFlashData = { error: string };

export const sessionContext = createContext<Session<Sessi
```

```ts
// src/middlewares/session.server.ts
import type { MiddlewareFunction } from "react-router";
import { getSession, commitSession } from "@/sessions.ser
import { sessionContext } from "@/contexts";

export const sessionMiddleware: MiddlewareFunction<Respor
  { request, context },
  next,
) => {
  const session = await getSession(request.headers.get("C
  context.set(sessionContext, session);

  const response = await next();

  // Persist any mutations made by loaders or actions
  response.headers.append("Set-Cookie", await commitSessi
  return response;
};
```

```ts
// In any loader — no cookie parsing needed
import { sessionContext } from "@/contexts";

export function loader({ context }: Route.LoaderArgs) {
  const session = context.get(sessionContext);
  const userId = session.get("userId");
  // ...
}
```

This eliminates duplicated session-parsing logic across routes and centralizes the `commitSession` call. The middleware owns the session lifecycle, loaders and actions simply read and write session data.

For details on how cookies and sessions fit into the broader state model alongside URL state, middleware context, and React primitives, see State Management.

Revalidation Control

By default, React Router automatically revalidates (re-executes) loaders after navigation events, form submissions, and actions to ensure data stays fresh. While this default behavior guarantees data

fine-grained control over when a route's loader should re-execute, enabling you to optimize performance by preventing redundant data fetching while maintaining data freshness where it matters.

Example: Product List with Filtering

This example shows a product listing page where query parameters control filters. The example uses the category, price range, and sort order query parameters. Because the loader already uses these query parameters to fetch filtered results, we don't need to revalidate when only the URL search parameters change. The loader naturally fetches the correct data on the next navigation.

```
1  // src/routes/products.tsx
2  import type { LoaderFunctionArgs, ShouldRevalidateFunctio
3
4  export async function loader({ request }: LoaderFunctionA
5    const url = new URL(request.url);
6    const category = url.searchParams.get("category") || "a
7    const minPrice = url.searchParams.get("minPrice") || "0
8    const maxPrice = url.searchParams.get("maxPrice") || "1
9    const sort = url.searchParams.get("sort") || "relevance
10
11   // Loader naturally handles query params - no revalidat
12   const products = await fetch(
13     `https://api.example.com/products?category=${category
14   ).then((r) => r.json());
15
16   return { products, filters: { category, minPrice, maxPr
17 }
18
19 export function shouldRevalidate({
20   currentUrl,
21   nextUrl,
22   actionStatus,
23   actionResult,
24 }: ShouldRevalidateFunctionArgs): boolean {
25   const currentPath = new URL(currentUrl).pathname;
26   const nextPath = new URL(nextUrl).pathname;
27
28   // Revalidate if navigating to a different route
29   if (currentPath !== nextPath) {
30     return true;
31   }
32
33   // Revalidate if an action modified product data (e.g.,
34   if (actionStatus === 200 && actionResult?.productsModif
35     return true;
36   }
37
38   // Don't revalidate for query param changes - loader ha
39   // This prevents redundant fetches when filters change
40   return false;
41 }
```

Tip

When your loader consumes URL search parameters to fetch data, returning `false` for query parameter changes

DID THIS ARTICLE SOLVE YOUR ISSUE?
Let us know so we can improve!

Share your feedback